

Create a DataFrame

```
import pandas as pd

data = {
    "Name": ["Anu", "Ravi", "Kiran", "Sita", "Rahul"],
    "Age": [22, 25, 21, 24, 27],
    "City": ["Rajahmundry", "Hyderabad", "Chennai", "Rajahmundry", "Hyderabad"],
    "Salary": [25000, 45000, 30000, 35000, 55000],
    "Department": ["IT", "HR", "IT", "Finance", "IT"]
}

df = pd.DataFrame(data)

print(df)
```

Output

	Name	Age	City	Salary	Department
0	Anu	22	Rajahmundry	25000	IT
1	Ravi	25	Hyderabad	45000	HR
2	Kiran	21	Chennai	30000	IT
3	Sita	24	Rajahmundry	35000	Finance
4	Rahul	27	Hyderabad	55000	IT

2. Filtering Data

Salary greater than 30000

```
result = df[df["Salary"] > 30000]
```

```
print(result)
```

Multiple conditions

```
result = df[
    (df["Salary"] > 30000) &
    (df["Department"] == "IT")
]
```

```
print(result)
```

OR condition

```
result = df[
    (df["City"] == "Hyderabad") |
```

```
(df["City"] == "Rajahmundry")  
]  
  
print(result)
```

Important

In Pandas:

&

means **AND**

|

means **OR**

Do not use:

and

or

inside Pandas conditions.

3. Filter Using **isin()**

Suppose we want employees from Hyderabad or Chennai.

```
result = df[df["City"].isin(["Hyderabad", "Chennai"])]  
  
print(result)
```

This is cleaner than writing multiple **OR** conditions.

4. Sorting Data

Sort salary ascending

```
df.sort_values("Salary")
```

Sort salary descending

```
df.sort_values("Salary", ascending=False)
```

Sort by multiple columns

```
df.sort_values(  
    ["Department", "Salary"],  
    ascending=[True, False]  
)
```

Meaning:

1. Department → A to Z
2. Salary → highest to lowest

5. Create a New Column

Suppose employees get a **10% bonus**.

```
df["Bonus"] = df["Salary"] * 0.10
```

```
print(df)
```

Now:

Salary	Bonus
25000	2500
45000	4500
30000	3000
35000	3500
55000	5500

6. Calculate Total Salary

```
df["TotalSalary"] = df["Salary"] + df["Bonus"]
```

```
print(df)
```

7. Conditional Column with **np.where()**

Let's classify employees based on salary.

```
import numpy as np
```

```
df["Level"] = np.where(
    df["Salary"] >= 40000,
    "Senior",
    "Junior"
)
```

```
print(df)
```

Example:

Salary	Level
25000	Junior
45000	Senior
30000	Junior
35000	Junior
55000	Senior

8. `groupby()` — VERY IMPORTANT 🔥

`groupby()` is one of the most important Pandas concepts for Data Analysts.

Suppose we want the **average salary by department**.

```
result = df.groupby("Department")["Salary"].mean()
```

```
print(result)
```

Example:

Department	
Finance	35000
HR	45000
IT	36666.67

9. Multiple Aggregations

```
result = df.groupby("Department")["Salary"].agg(
    ["count", "sum", "mean", "min", "max"]
)
```

```
print(result)
```

You can get:

- Employee count
- Total salary
- Average salary
- Minimum salary
- Maximum salary

in one operation.

10. Group By City

```
result = df.groupby("City")["Salary"].mean()
```

```
print(result)
```

This answers a real business question:

What is the average salary in each city?

11. Group By Multiple Columns

```
result = df.groupby(
    ["City", "Department"]
)["Salary"].mean()
```

```
print(result)
```

This is useful for analyzing data at a more detailed level.

12. `value_counts()`

Suppose we want to know how many employees are in each department.

```
print(df["Department"].value_counts())
```

Example:

```
IT      3
HR      1
Finance 1
```

City-wise employee count

```
print(df["City"].value_counts())
```

13. Missing Values

Let's create missing data.

```
data = {  
    "Name": ["Anu", "Ravi", "Kiran", "Sita"],  
    "Age": [22, None, 21, 24],  
    "Salary": [25000, 45000, None, 35000]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

Find missing values

```
print(df.isnull())
```

Count missing values

```
print(df.isnull().sum())
```

Example:

```
Name    0  
Age      1  
Salary   1
```

14. Fill Missing Values

Fill Age with average age

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

Fill Salary with 0

```
df["Salary"] = df["Salary"].fillna(0)
```

15. Remove Missing Rows

```
df.dropna()
```

This removes rows containing missing values.

16. Remove Duplicate Data

```
df.drop_duplicates()
```

For a particular column:

```
df.drop_duplicates(subset=["Name"])
```

17. Rename Columns

```
df.rename(  
    columns={  
        "Salary": "MonthlySalary",  
        "Age": "EmployeeAge"  
    },  
    inplace=True  
)
```

```
print(df)
```

18. `loc[ ]`

`loc` is used for **label-based selection**.

```
print(df.loc[0])
```

Select specific columns:

```
print(df.loc[:, ["Name", "Salary"]])
```

Filter rows:

```
result = df.loc[df["Salary"] > 30000]
```

```
print(result)
```

19. `iloc[ ]`

`iloc` works using **position/index number**.

```
print(df.iloc[0])
```

First 3 rows:

```
print(df.iloc[0:3])
```

First 3 rows and first 2 columns:

```
print(df.iloc[0:3, 0:2])
```

Remember

loc → labels / conditions

iloc → positions

20. Real-Time Data Analyst Example

Imagine an employee dataset:

```
data = {  
    "Employee": ["A", "B", "C", "D", "E", "F"],  
    "Department": ["IT", "HR", "IT", "Sales", "HR", "IT"],  
    "Salary": [40000, 30000, 50000, 35000, 32000, 60000],  
    "Experience": [2, 1, 4, 3, 2, 6]  
}
```

```
df = pd.DataFrame(data)
```

Question 1: Employees earning above ₹40,000

```
df[df["Salary"] > 40000]
```

Question 2: Average salary

```
df["Salary"].mean()
```

Question 3: Highest salary

```
df["Salary"].max()
```

Question 4: Employee with highest salary

```
df.loc[df["Salary"].idxmax()]
```

Question 5: Average salary by department

```
df.groupby("Department")["Salary"].mean()
```

Question 6: Number of employees by department

```
df["Department"].value_counts()
```

Question 7: Employees with more than 3 years experience

```
df[df["Experience"] > 3]
```

Question 8: Add annual salary

```
df["AnnualSalary"] = df["Salary"] * 12
```



Level 3 Important Functions

Function	Purpose
<code>df[condition]</code>	Filter
<code>isin()</code>	Multiple-value filtering
<code>sort_values()</code>	Sorting
<code>groupby()</code>	Group analysis
<code>agg()</code>	Multiple calculations
<code>value_counts()</code>	Frequency/count
<code>isnull()</code>	Find missing values
<code>fillna()</code>	Fill missing values
<code>dropna()</code>	Remove missing values
<code>drop_duplicates()</code>	Remove duplicates
<code>rename()</code>	Rename columns
<code>loc[]</code>	Label-based selection

<code>iloc[]</code>	Position-based selection
<code>np.where()</code>	Conditional column

Level 3 Practice

Create this DataFrame:

```
data = {  
    "Name": ["A", "B", "C", "D", "E", "F"],  
    "City": ["Rajahmundry", "Hyderabad", "Chennai",  
             "Hyderabad", "Rajahmundry", "Chennai"],  
    "Department": ["IT", "HR", "IT", "Finance", "IT", "HR"],  
    "Salary": [25000, 40000, 35000, 45000, 50000, 30000],  
    "Experience": [1, 3, 2, 5, 6, 2]  
}  
  
df = pd.DataFrame(data)
```

Solve these without looking at the answers:

1. Find employees with salary greater than ₹35,000.
2. Find employees from Hyderabad.
3. Find employees from Hyderabad **and** salary greater than ₹35,000.
4. Sort employees by salary from highest to lowest.
5. Find average salary.
6. Find average salary department-wise.
7. Find maximum salary department-wise.
8. Count employees in each city.
9. Create an `AnnualSalary` column.
10. Create an `ExperienceLevel` column:
 - Experience $\geq 5 \rightarrow$ "Senior"
 - Experience $< 5 \rightarrow$ "Junior"